合併排序與分治入門

divide-and-conquer把問題分為嚴格更小的同類問題，遞迴求解，再合併答案。只說「分兩半」不夠：須定邊界、base case、合併前提及結果。merge sort取A[p..r]，長m=r−p+1；p>r表示空區間。

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

合法非空呼叫要求1≤p≤r≤n；整陣列呼叫(1,n)，n=0時(1,0)直接返回。左右長分別ceil(m/2)、floor(m/2)，m≥2時都小於m。q寫成差值式避免p+r溢出的常見實作風險，但仍須確保合法索引範圍可表達。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

不依賴額外key值的MERGE

前提$1 \le p \le q < r \le n$，兩半已穩定排序。L、R是**獨立副本**，不能是會被覆寫A影響的alias。
$len(L)=q-p+1$，$len(R)=r-q$。

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key
            A[k] = L[i]; i = i+1
        else
            A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相等先取L，因原輸入左半records全先於右半，穩定性得以延續。只檢查長度不需key域之外的∞。

MERGE的保存、順序與終止證明

主迴圈前，k−p=(i−1)+(j−1)；A[p..k−1]恰是已消耗的L[1..i−1]、R[1..j−1]之穩定合併，已排序且其中每個key≤所有未消耗key。其餘未消耗records仍在獨立L/R中。

初始化空輸出成立。若兩邊非空，各自首筆就是各邊最小未消耗值；選較小者（等值選左）恰為全體下一筆。消耗對應一筆、k加1，保持長度等式、保存、順序與穩定性。未消耗總數每次減1，故終止。

退出主迴圈時至少一邊耗盡；另一邊已排序且不小於輸出尾，所以逐筆接上保留同一不變量。最後i=len(L)+1、j=len(R)+1、k=r+1，全部m筆恰輸出一次。若不複製L/R，A[k]可能覆寫尚未讀的輸入；證明所需的剩餘資料就消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有重複key的完整合併

L=[1a,4b,4c]、R=[2d,4e,6f]，原輸入左段先於右段。表列主迴圈每次比較；i,j為比較前的1-based位置。

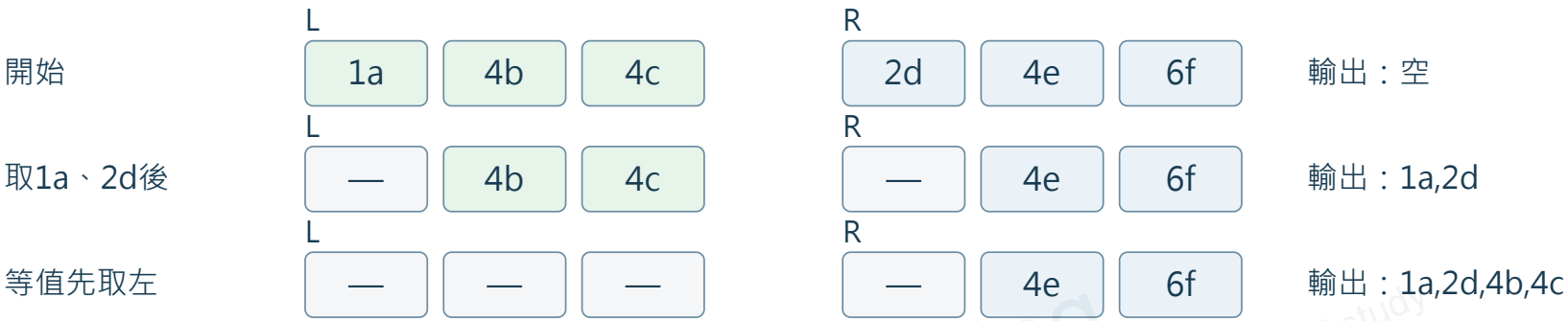
i,j	比較	取出後的輸出prefix
1,1	1a≤2d	1a
2,1	4b>2d	1a,2d
2,2	4b≤4e	1a,2d,4b
3,2	4c≤4e	1a,2d,4b,4c
4,2	L耗盡，不再比較key	接上4e,6f

共4次key比較、6次輸出寫入，另有6筆copy進L/R。若最後兩筆交錯耗盡，最多m−1=5次key比較；「MERGE是Θ(m)」不表示每次恰做m次key比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有重複key的完整合併 （續）

消耗一筆，輸出一筆；兩邊副本不會被A回寫破壞



L耗盡 → 依序接4e、6f。相同key的4b、4c、4e保持原本從左至右的先後。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sentinel是有前提的另一種表示

若使用sentinel，可在L/R尾各加一筆key=∞，固定m次從較小首筆取出，不用每次檢查耗盡。但∞必須嚴格大於每個合法key，且不是合法record。將最大合法整數當∞，遇到同值輸入就可能錯取sentinel，甚至下一步越界。

例如合法key允許+∞，L=[+∞]、R=[+∞,+∞]，加上相同∞哨兵後「等值選左」可能連取左真record、左sentinel，第三次便讀超過L。這不是正確的域外標記。

本章長度檢查版對任意total-order key域都可用，代價只是明確處理耗盡。若改為tagged sentinel，使它的排序順位嚴格高於所有real record，也須保證sentinel絕不輸出成資料。

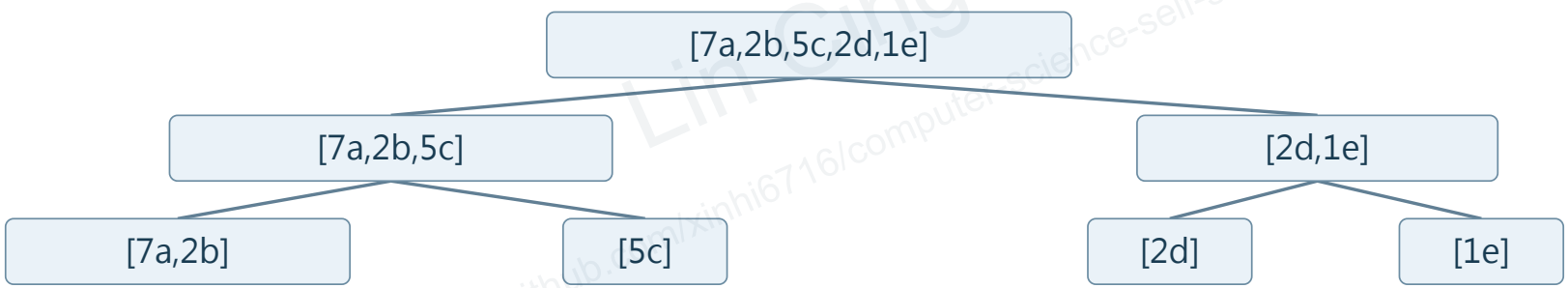
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從局部合併推到整個排序

對區間長m作strong induction。m≤1已排序且不變。m≥2時兩個子問題都嚴格較小，歸納假設使兩半各自穩定排序且保持各半全部records；MERGE前提成立。前頁證明使整段穩定排序，區間外未改，歸納完成。

終止由子問題長度遞減至0/1保證；正確性不依賴n是2的冪。輸入[7a,2b,5c,2d,1e]分為長3、2，再分2/1與1/1；各節點合併結果如下圖。左側2b與右側2d相等，根合併仍先2b。

五筆record：向下分割，向上回傳穩定排序結果



自底向上： [2b,7a] → [2b,5c,7a]；右半 → [1e,2d]

根合併： [1e,2b,2d,5c,7a]；相等key2：左2b先於右2d

左長3、右長2；單筆直接返回。圖省略[7a,2b]再分為兩個單筆的最後一層。

遞迴式的成本不能漏掉合併

對m≥2，分割計q為Θ(1)，兩個遞迴呼叫成本相加，MERGE複製m筆並輸出m筆，為Θ(m)，故

T(m)=T(ceil(m/2))+T(floor(m/2))+Θ(m)，T(0),T(1)=Θ(1)。

先對m=2^h採精確代理式U(1)=d、U(m)=2U(m/2)+cm（c,d>0）。深度ℓ=0..h−1有2^ℓ個internal nodes，每個成本cm/2^ℓ，該層總cm；h層合cmh。最底m個葉各d，故U(m)=cm lg m+dm。

真成本每internal node夾於c_low・size與c_high・size、葉成本也有固定上下界，將兩者各自代入便得到Θ(m lg m)，而非假設同一c恰代表所有指令和base case。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

任意長度的界與一個六筆成本表

平衡拆分到深度ℓ後，各未終止區間長度至多ceil(m/2^ℓ)，所以height≤ceil(lg m)。同一深度的區間互不重疊，總長至多m，給internal成本O(m lg m)。

在深度ℓ<floor(lg m)，每段至少floor(m/2^ℓ)≥2，尚無葉；各段總長恰m，給floor(lg m)層各Ω(m)。m≥2時floor(lg m)≥(lg m)/2，故下界Ω(m lg m)。這是任意m的雙邊證明。

m=6的深度	問題長度	internal長度總和
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1（另兩葉已在上層）	0

本例copy進buffers與回寫A各16筆，不是把每一層都誤算成6筆；base case共6個。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

峰值空間不是所有配置次數的總和

本版MERGE在兩次遞迴返回後才建立L/R，完成合併即可釋放；單次長m的buffers合m筆。子呼叫的buffers不與父MERGE buffers同時存活，因此峰值輔助空間Θ(n)，另有Θ(lg n)呼叫stack，合仍Θ(n)（n≥2）。

整趟累計buffer配置量可以是Θ(n lg n)，但它不是同時需要的記憶體。若實作保留每個已完成呼叫的buffer不釋放，就換了生命期模型；不能套用上述峰值結論。也可預先配置n筆scratch反覆重用，並保留同樣的非alias前提。

insertion sort只需key、i、j等常數額外word，較省空間；merge sort在本實作保證Θ(n lg n)時間但非in-place。兩者都穩定，前提分別是strict >右移與等值先取左。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study